

Technical Assessment Brief: Order Management System

Stack: Next.js (App Router) · Supabase (Postgres, Auth, Storage) · Stripe (test mode) · Vercel

1. Purpose

Build a small but production-shaped e-commerce system with three surfaces: a customer-facing ordering front end, a backend for stock management, and an admin panel for managing orders. The goal is to assess your ability to ship a secure, well-structured full-stack application end to end, not to build a polished product.

We are evaluating judgement as much as output: data modelling, security boundaries, payment correctness, and how you handle the edge cases you'll inevitably hit.

Timebox: Estimate 12–20 hours of work. Submit within 7 calendar days of receiving this brief.

2. Required stack

Layer	Requirement
Framework	Next.js 14+ (App Router), TypeScript
Database	Supabase Postgres with Row Level Security enabled on every table
Auth	Supabase Auth (email/password minimum). Clerk or Auth.js accepted if you justify the choice and still enforce RLS via JWT claims
Payments	Stripe in test mode, using Stripe Checkout or Payment Elements, with webhook-driven order confirmation
Hosting	Vercel, deployed from a Git repository with a working production URL
Storage	Supabase Storage for product images (optional but credited)

Use of an ORM (Drizzle, Prisma) or the Supabase client directly is your choice. UI library is your choice.

3. Scope

3.1 Customer front end (storefront)

- Product listing page showing name, image, price, and current availability
- Product detail page
- Cart (client-side state is fine; persistence across sessions is a bonus)
- Checkout flow that creates an order and redirects to Stripe
- Order confirmation page after successful payment
- "My orders" page for authenticated customers showing status history
- Out-of-stock products must not be purchasable; quantity selected cannot exceed available stock

3.2 Backend: stock management

- Products table with SKU, name, description, price (stored in minor units, e.g. pence/cents), image, and active flag
- Inventory model tracking on-hand quantity per product
- Stock must be **reserved or decremented atomically** at the point of order creation, and released if payment fails or the checkout session expires
- Concurrency must be handled: two customers cannot both buy the last unit. Document your approach (row locking, `SELECT ... FOR UPDATE`, database function, optimistic check, etc.)
- A stock movement log (reason, delta, reference, timestamp) so every change is auditable

3.3 Admin panel

- Accessible only to users with an `admin` role
- Product CRUD: create, edit, deactivate, upload image, adjust stock with a reason
- Order list with filtering by status and search by order ID or customer email
- Order detail: line items, customer, payment reference, status timeline
- Order status transitions: `pending_payment` → `paid` → `fulfilled` and `pending_payment` → `cancelled`, `paid` → `refunded`. Invalid transitions must be rejected server-side
- Issue a refund from the admin panel via the Stripe API (full refund is sufficient)

3.4 Authentication and authorisation

- Sign up, sign in, sign out, password reset
- Two roles: `customer` and `admin`. Role must be stored in the database (or JWT custom claims), never inferred client-side
- RLS policies such that:
 - Customers can read only their own orders
 - Customers cannot read or modify inventory, other users' data, or admin-only fields
 - Admins can read and write products, inventory, and all orders
 - The anonymous role can read active products only
- Server-side route protection for `/admin/*`. Middleware alone is not sufficient; server actions and API routes must also verify role
- Service-role key must never be exposed to the browser

3.5 Payments (Stripe test mode)

- Create a Stripe Checkout Session (or PaymentIntent) from a server action, with amount calculated **server-side from database prices**, never from client-submitted totals
 - Store the Stripe session/intent ID against the order
 - Implement a webhook endpoint that:
 - Verifies the Stripe signature
 - Handles `checkout.session.completed` / `payment_intent.succeeded` to mark the order `paid`
 - Handles `checkout.session.expired` / `payment_intent.payment_failed` to release stock and mark the order `cancelled`
 - Is **idempotent**: replaying the same event must not double-process
 - The order must only become `paid` via the webhook, not via the client redirect
 - Support refund via `refunds.create`, with the resulting webhook updating order status
-

4. Data model

You are expected to design this yourself, but at minimum we expect tables along the lines of:

- `profiles` (user id, role, display name)
- `products`
- `inventory` or an inventory column with a movement log
- `orders`
- `order_items` (with price snapshotted at time of order)
- `stock_movements`
- `stripe_events` or equivalent for idempotency

Provide migrations as SQL files (or Supabase migration folder). We should be able to recreate the schema from your repository.

5. Non-functional requirements

- **Security:** RLS on all tables, no service key in client bundles, webhook signature verification, no trust of client-submitted prices or quantities

- **Correctness:** No overselling. No order marked paid without a verified Stripe event
- **Error handling:** Meaningful errors on failed checkout, failed webhook, and stock conflicts. No silent failures
- **Code quality:** Typed end to end, sensible folder structure, no dead code. Lint passes
- **Environment:** `.env.example` provided. Secrets never committed
- **Accessibility:** Basic keyboard navigation and labelled form fields

Not required: unit test coverage targets, i18n, multi-currency, shipping calculation, tax, email notifications, real payment processing.

6. Deliverables

1. **Git repository** (GitHub, public or shared with us) containing:
 - Source code
 - Supabase migrations and any seed script
 - `README.md` covering local setup, environment variables, how to run the Stripe CLI webhook forwarder locally, and how to seed an admin user
 2. **Live deployment on Vercel** with a working URL
 3. **Test credentials:** one customer account and one admin account
 4. **A short technical write-up** (500–1000 words, in the README or a separate `DECISIONS.md`) covering:
 - Your approach to stock reservation and concurrency
 - How you achieved webhook idempotency
 - Your RLS policy design and one policy you found tricky
 - What you would do differently with another week
 - Anything you cut for time and why
 5. **Stripe test evidence:** we will place a test order using card `4242 4242 4242 4242` and a declined card `4000 0000 0000 0002`, and we will replay a webhook event using the Stripe CLI
-

7. Submission

Reply to the email that sent you this brief with the repository link, live URL, and test credentials. Questions about the brief are welcome at any point; asking good clarifying questions is a positive signal, not a negative one.